Orbital Advection Across Azimuthal Level Jumps in the AthenaK Shearing Box

Implementation Document

`athenak-multigrid` tree, branch `fargo-amr`
parent commit `d3edb948` (branch `dust-multigrid`)

October 28, 2026

Abstract

With orbital advection (FARGO) on, AthenaK's shearing box could be refined only in complete azimuthal rings: every x_2 neighbor of a MeshBlock had to be at its own level. For clump-following adaptive refinement that is the dominant cost — a fragment in the $64H$ SC14 box refines a $64H$-long ring at every level. This document describes how the cell-centered orbital remap was extended so that an x_2 neighbor may be one level coarser or finer. Each block still remaps on its own grid; the neighbor's rows are restricted (from a finer neighbor) or prolongated (from a coarser one) to the block's resolution before the remap. The remap is rewritten in flux form, and an orbital-advection flux correction makes it exactly conservative across the level jump. A timestep limit keeps the shift within the immediate neighbor. The ring policy becomes an option. Uniform and ring-refined runs are bit-identical to the parent code. For MHD (branch `fargo-amr-mhd`) orbital advection now runs on refined meshes whose refinement comes in complete x_2 rings: the effective EMFs of the face-field remap are corrected at level jumps like the stage EMFs, which keeps the two copies of every face shared by a fine and a coarse block equal. Face fields across x_2 level jumps (MHD with partial-x_2 refinement) are the next stage. The tests are in the companion `fargo_amr_validation.pdf`.

Contents

1 Scope and how to read this **3**
1.1 Files touched . . . 3

2 Background **3**
2.1 Orbital advection in AthenaK . . . 3
2.2 The annular policy and what it cost . . . 4
2.3 Prior art: Athena++ . . . 4

3 Design **4**
3.1 Three kinds of neighbor . . . 4
3.2 Buffers and messages . . . 5
3.3 Unpacking: index maps . . . 6

4 The orbital-advection flux correction **6**
4.1 The remap in flux form . . . 6
4.2 Why the two sides disagree, and the correction . . . 6

5 How far the shift may reach **7**
5.1 Rows available, and the guard . . . 7
5.2 The timestep limit . . . 7
5.3 The old buffer depth . . . 8

6 Policy and control flow **8**
6.1 The refinement policy . . . 8
6.2 Task sequence of the last RK stage . . . 8
6.3 A test criterion: the moving box . . . 8

7 Magnetic fields: MHD on rings **8**
7.1 How the face-field remap works . . . 8
7.2 What goes wrong at a level jump, even with rings . . . 9
7.3 The correction . . . 9
7.4 Policy . . . 10
7.5 A diagnostic: shared-face consistency . . . 10

8 Input-file interface **10**
8.1 Parameters . . . 10
8.2 Reference inputs . . . 10

9 Guard reference **11**

10 Limitations and next steps **11**

1 Scope and how to read this

This document describes mechanisms — data layout, index maps, call sites, guards — not results; those are in `fargo_amr_validation.pdf`. It builds on the shearing-box refinement work documented in `multigrid_sb_implementation.pdf` (Phase 0 there: the refinement policy, per-level orbital advection, the non-FARGO mode, AMR rebuilds of the shear state). That document now carries short pointers here where its statements about the ring policy are out of date.

1.1 Files touched

File	Change
`src/shearing_box/orbital_advection_cc.cpp`	Rewritten: pack (same, restricted, coarse-box messages), unpack with prolongation, remap fluxes G at the block faces, `SendFluxCC`, `RecvAndCorrectFluxCC`, `RowsAvailable`, `MaxTimeStep`
`src/shearing_box/orbital_advection.hpp`	`OrbitalAdvectionCC` buffers, sizes, MPI requests; virtual `InitRecv`/`ClearRecv`/`ClearSend`
`src/shearing_box/orbital_advection_tasks.cpp`	CC overrides of the three MPI functions (per-sub-face requests, flux receives)
`src/hydro/hydro{.hpp,_tasks.cpp}`	Tasks `SendFlx_OA`, `RecvFlx_OA`
`src/dust/dust{.hpp,_tasks.cpp}`	Same two tasks in the dust module's copy of the hydro task tail
`src/mesh/mesh.cpp`	Orbital-advection timestep limit in `Mesh::NewTimeStep`; policy warning text
`src/mesh/build_tree.cpp`	Ring check (I2) only with `oa_annular`
`src/mesh/mesh_refinement.{cpp,hpp}`	Ring-wise flag sync only with `oa_annular`
`src/hydro/hydro.cpp`, `src/mhd/mhd.cpp`	Comments
`src/pgen/tests/shwave.cpp`	Optional x_2 window for the moving-ring criterion (a moving box)
MHD on rings (branch `fargo-amr-mhd`)	
`src/shearing_box/orbital_advection_fc.cpp`	EMF correction of the face-field remap (`SendEMF_OA`, `RecvEMFAndUpdateB_OA`, `UpdateB`); buffer depth `nrowb` $\leq n_{x2}$; EMF arrays sized for AMR
`src/shearing_box/orbital_advection{.hpp,_tasks.cpp}`	FC overrides of `InitRecv`/`ClearRecv`/`ClearSend`
`src/mhd/mhd{.cpp,.hpp,_tasks.cpp}`	Guard relaxed to rings; tasks `SendE_OA`, `RecvE_OA`, `SendFlx_OA`, `RecvFlx_OA`
`src/pgen/tests/swing.cpp`	History columns `bmis`, `bnrm`: shared-face consistency

2 Background

2.1 Orbital advection in AthenaK

In the shearing box the background flow is $v_y = -q\Omega_0 x$. With orbital advection the state carries only the deviation from it, and the background transport is applied as a separate step: at the end of each time step (last RK stage, after the source terms) every cell is shifted in x_2 by

$$\Delta y = -q\Omega_0\, x_1\, \Delta t = (J + \epsilon)\, \Delta x_2, \qquad J = \mathrm{trunc}(\Delta y/\Delta x_2), \quad |\epsilon| < 1, \tag{1}$$

per x_2 column (fixed x_1, x_3). The integer part J is a copy, the fractional part a conservative remap with upwind "fluxes" F from a PLM or PPM reconstruction (`remap_fluxes.hpp`):

$$a_j^{\text{new}} = a_{j-J} - \left(F_{j-J+1/2} - F_{j-J-1/2}\right). \tag{2}$$

`PackAndSendCC` sends the rows next to each x_2 face to the neighbor; `RecvAndUnpackCC` builds an extended column (own cells plus neighbor rows on both sides) in team scratch memory and applies Eq. (2). The tasks `SendU_OA`/`RecvU_OA` sit between the source terms and `RestrictU` of the last stage, so the shifted state then goes through the normal boundary exchange.

2.2 The annular policy and what it cost

The old pack used the same indices on both sides of a face ("neighbor must always be at same level") and read only neighbor slots 8 and 12, the first sub-face of each x_2 face. A coarser neighbor, which sits in sub-face slot 9–11 or 13–15, was not even read. Invariant I2 (complete x_2 rings at every level, with a ring-wise OR/AND of AMR flags) kept every x_2 neighbor at the same level. In the SC14 clump-following AMR runs ($64H \times 64H \times 16H$, 8^3 blocks) a fragment's refinement therefore covered the whole $64H$ azimuth at every level, reaching ∼28 000 blocks at $\beta = 4$ and 70–100 000 in the first $\beta = 3$ attempt.

2.3 Prior art: Athena++

Athena++'s orbital advection (T. Ono) supports refinement. Each block remaps on its own grid with an extended column; a finer neighbor sends restricted data, a coarser one a box of its own cells that the receiver prolongates with the standard limited-linear operator; a timestep limit keeps the shift within one neighbor block. Orbital advection runs as a separate stage with its own boundary cycle, so the prolongation uses fresh ghost cells. There is no flux correction: mass is conserved across level jumps only to truncation error. The design here follows the same pattern, adds the flux correction, and fuses the remap into the last RK stage as before.

3 Design

3.1 Three kinds of neighbor

2:1 balance guarantees that an x_2 face neighbor is at most one level away. Seen from a block on one of its x_2 faces (`OAFaceKind`):

Kind	Neighbor slots	What the block receives
same level	one, sub-face 0	the `nrow` rows next to the face, copied
finer	four (one per (x_1, x_3) quadrant)	from each fine block: its 2 `nrow_fc` rows next to the face, restricted (2^3 averages) to `nrow_fc` coarse rows over its quadrant
coarser	one, sub-face $q = (l_{x1} \bmod 2) + 2(l_{x3} \bmod 2)$	a box of coarse cells over the block's quadrant, prolongated by the receiver

In every case the column handed to the remap is at the receiver's own resolution, so Eq. (2) and the reconstruction kernels are unchanged. The shift of a fine column uses the fine cell center, of a coarse column the coarse one; because Δy is linear in x_1, the two fine x_1 sub-columns of a coarse column move on average exactly as the coarse column does.

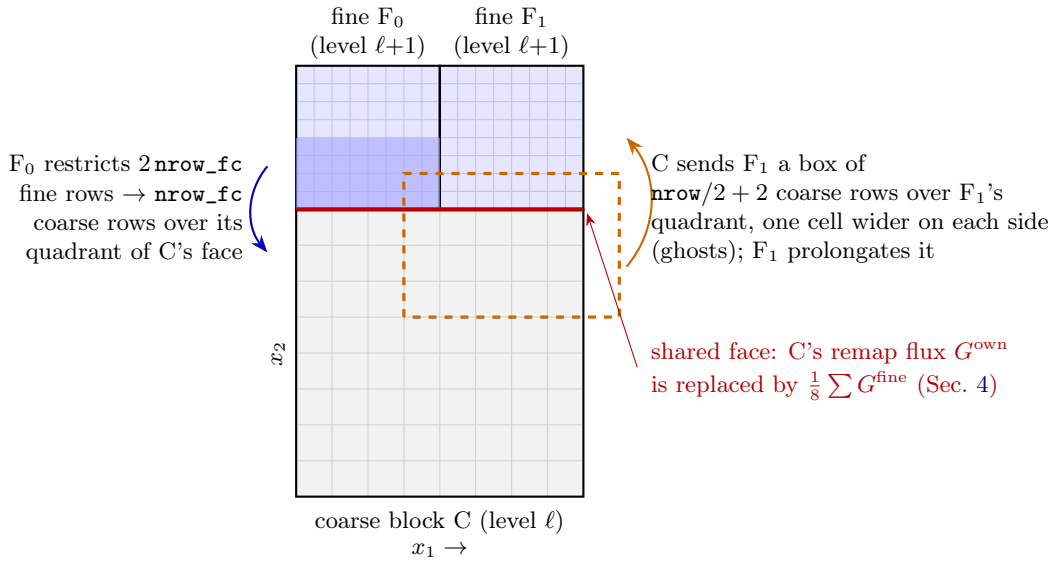


Figure 1: The two cross-level messages on an x_2 face, in the x_1–x_2 plane (the two further fine blocks of the face are behind, in x_3). Each block remaps on its own grid; the neighbor's rows arrive restricted or as a coarse box to prolongate. After the remap, the fine blocks send their face fluxes to C.

3.2 Buffers and messages

Buffers are flat arrays per (MeshBlock, face): `sbuf_oa`, `rbuf_oa` of shape $(n_{\mathrm{mb}}, 2, \texttt{sz_face})$. Face $f = 0$ is the inner x_2 face (neighbor slots 8–11), $f = 1$ the outer (12–15). The message for sub-face q (slot $8 + 4f + q$) starts at offset $q \cdot n_{\mathrm{var}} \cdot s$, where s is the per-variable size of that message kind; variable v starts at $v\,s$, and inside a variable the cells run with i fastest and rows in ascending j:

Message	Cells per variable	Rows (sender)
same level	$\texttt{sz_same} = n_{x3} \cdot \texttt{nrow} \cdot n_{x1}$	$j_s \ldots j_s{+}\texttt{nrow}{-}1$ ($f{=}0$) or $j_e{-}\texttt{nrow}{+}1 \ldots j_e$
fine → coarse	$\texttt{sz_fc} = \frac{n_{x3}}{2} \cdot \texttt{nrow_fc} \cdot \frac{n_{x1}}{2}$	$2\,\texttt{nrow_fc}$ fine rows next to the face, restricted
coarse → fine	$\texttt{sz_cf} = (\frac{n_{x3}}{2}{+}2) \cdot \texttt{nrow_cf} \cdot (\frac{n_{x1}}{2}{+}2)$	$j_s{-}1 \ldots j_s{+}\texttt{nrow}/2$ or $j_e{-}\texttt{nrow}/2 \ldots j_e{+}1$

with $\texttt{nrow_fc} = \min(\texttt{nrow}, n_{x2}/2)$ and $\texttt{nrow_cf} = \texttt{nrow}/2 + 2$, and

$$\texttt{sz_face} = n_{\mathrm{var}} \max\left(\texttt{sz_same},\ 4\max(\texttt{sz_fc}, \texttt{sz_cf})\right).$$

The coarse box of quadrant (f_x, f_z) spans $i = i_s + f_x n_{x1}/2 - 1 \ldots i_s + (f_x{+}1)n_{x1}/2$ and the same in k. Its outermost cells (the row across the face and the transverse layer at the block edge) are ghost cells.

A message lands in the receiver's buffer at the receiver's sub-face offset, read from `nghbr.dest`. Senders on the same rank write straight into the receiver's buffer; otherwise the message goes through `sbuf_oa` and `MPI_Isend` with tag $(\texttt{dest} \ll \texttt{NUM_BITS_LID}) \,|\, \texttt{lid}$ on `comm_orb_advect`, as the other boundary exchanges do. Receives are posted per sub-face in `OrbitalAdvectionCC::InitRecv`, with the size of the kind seen from the receiver; MPI requests are kept per (block, sub-face), $8n_{\mathrm{mb}}$ of each.

3.3 Unpacking: index maps

The extended column has $n_{x2} + 2\,\texttt{nrow}$ cells; the block's own cells start at $j_f = \texttt{nrow}$. The $\texttt{nrow}$ rows beyond face f are numbered $t = 0 \ldots \texttt{nrow} - 1$ in ascending j (so on the inner face $t = \texttt{nrow} - 1$ touches the block, on the outer face $t = 0$ does). Row t is loaded by:

- **same level:** the received row t.
- **finer:** quadrant $f_x = \lfloor (i - i_s)/(n_{x1}/2) \rfloor$, f_z likewise; coarse row $t_c = t - (\texttt{nrow} - \texttt{nrow_fc})$ on the inner face, $t_c = t$ on the outer. Rows with t_c outside $[0, \texttt{nrow_fc})$ are never read (guarded, Sec. 5).
- **coarser:** with the distance d from the face ($d = \texttt{nrow} - 1 - t$ on the inner face, $d = t$ on the outer), the coarse cell is at box row $jj = \texttt{nrow}/2 - \lfloor d/2 \rfloor$ (inner) or $\lfloor d/2 \rfloor + 1$ (outer), box column $ii = \lfloor (i - i_s)/2 \rfloor + 1$, box layer kk likewise. The value is

$$c + s_1\,\delta_1 + s_2\,\delta_2 + s_3\,\delta_3, \qquad \delta_n = \tfrac{1}{8}\left[\operatorname{sgn}(\Delta_n^-) + \operatorname{sgn}(\Delta_n^+)\right] \min(|\Delta_n^-|, |\Delta_n^+|), \tag{3}$$

  the `ProlongCC` formula ($\Delta_n^\pm$ the one-sided differences of the coarse box in direction n), with $s_1 = -1$ for even $i - i_s$ and $+1$ for odd, s_3 likewise in k, and $s_2 = +1$ for the fine row in the upper half of the coarse cell (d even on the inner face, odd on the outer).

The ghost cells used for the slopes hold the state of the previous stage's exchange: for RK2 they differ from the updated state by $O(\Delta t^2)$, and they only enter minmod-limited slopes. The remap shifts the coarse-box values like any other, and their mean over each coarse cell is the coarse value, so the prolongation conserves.

4 The orbital-advection flux correction

4.1 The remap in flux form

Eq. (2) telescopes: with

$$G_{j-1/2} = \begin{cases} F_{j-J-1/2} + \displaystyle\sum_{l=j-J}^{j-1} a_l, & J \geq 0, \\ F_{j-J-1/2} - \displaystyle\sum_{l=j}^{j-J-1} a_l, & J < 0, \end{cases} \qquad a_j^{\text{new}} = a_j - \left(G_{j+1/2} - G_{j-1/2}\right), \tag{4}$$

$G_{j-1/2}$ is the total amount (per cell volume) moved in $+x_2$ through face $j - 1/2$: the $|J|$ whole cells it passes plus the fractional flux. In the unpack kernel, after the fractional fluxes are known, one thread per column evaluates G at the block's two x_2 faces and stores it in $\texttt{gflx}(m, f, v, k, i)$ (multilevel meshes only).

4.2 Why the two sides disagree, and the correction

Across a level jump the coarse block remaps restricted fine data with its column velocity, and each fine block remaps prolongated coarse data with its own. Both approximate the same transport, but not identically, so without correction the mass that leaves one side is not the mass that enters the other (the mass drift is 10^{-7}–10^{-6} in the tests). The fine side is taken as the better estimate, as in

the hydro flux correction. A coarse column covers 2×2 fine columns and the volume ratio is $1/8$, so the amount through the coarse face is

$$G^{\text{fine}} = \tfrac{1}{8} \sum_{2\times 2} G_f. \tag{5}$$

`SendFluxCC` (task `SendFlx_OA`, after `RecvU_OA`) computes this on every block whose x_2 neighbor is coarser and sends it ($\frac{n_{x3}}{2} \cdot \frac{n_{x1}}{2}$ values per variable; separate communicator `comm_orb_flx`). `RecvAndCorrectFluxCC` (task `RecvFlx_OA`) replaces the coarse block's own flux in the row next to the face:

$$a_{j_s} \mathrel{+}= G^{\text{fine}} - G^{\text{own}} \quad \text{(inner face)}, \qquad a_{j_e} \mathrel{+}= G^{\text{own}} - G^{\text{fine}} \quad \text{(outer face)}. \tag{6}$$

The two faces are separate kernels. The receives are posted with the data receives in `InitRecv`, waited for in `ClearRecv`. `<shearing_box> oa_flux_correct = false` switches the correction off (for tests). The dust module builds its own copy of the hydro task tail (`dust_tasks.cpp`); the two tasks are inserted there too, or the posted flux receives would never be matched.

5 How far the shift may reach

5.1 Rows available, and the guard

The PPM reconstruction of the upwind cell reads two cells on either side of it, so the remap of a block with integer shift J reads $|J| + 3$ rows from its upstream neighbor (and $\max(0, 3 - |J|)$ from the other). All of them must come from the immediate x_2 neighbor. `RowsAvailable`(m) returns

$$n_{\text{avail}} = \begin{cases} \texttt{nrow} & \text{neither } x_2 \text{ neighbor finer,} \\ \min(\texttt{nrow}, n_{x2}/2) & \text{a finer } x_2 \text{ neighbor (a fine block is } n_{x2}/2 \text{ coarse rows).} \end{cases} \tag{7}$$

`nrow` defaults to $\min(n_{x2}, \texttt{maxjshift} + 3)$ (rounded up to even on multilevel meshes, so that it is a whole number of coarse rows) and can be set with `<shearing_box> oa_nrow` ($\le n_{x2}$). `RecvAndUnpackCC` checks every step, with this step's Δt and each block's $\max|x_1|$, that $\lfloor q\Omega_0 \Delta t \max|x_1|/\Delta x_2 \rfloor + 3 \le n_{\text{avail}}$, and aborts naming the block otherwise.

5.2 The timestep limit

The guard alone would end a long AMR run the first time the mesh put a fine block next to a coarse one at large $|x_1|$. `OrbitalAdvectionCC::MaxTimeStep` therefore returns

$$\Delta t_{\text{OA}} = \min_m \; 0.999 \, \frac{(n_{\text{avail},m} - 2)\, \Delta x_{2,m}}{q\Omega_0 \max|x_1|_m}, \tag{8}$$

which `Mesh::NewTimeStep` takes into the cycle timestep (not multiplied by the CFL number), for hydro and MHD alike. The driver calls `Mesh::NewTimeStep` after the AMR update, so the limit is evaluated on the mesh the next step will use. It binds only when shifts are large. On 8^3 blocks a coarse block next to a finer x_2 neighbor allows one coarse cell per step: at $|x| = 32H$ and 4 cells per H on the coarse side, $\Delta t \le 0.0104/\Omega_0$ at the root level and $2^{-\ell}$ times that at level ℓ. In the SC14 runs the hydro timestep is well below this at the root level; on refined levels the two can be comparable.

5.3 The old buffer depth

The previous code allocated and packed `ng` + `maxjshift` rows per face, with `maxjshift` = $\lfloor \mathrm{CFL}\, q\Omega_0 \max|x_1| \rfloor + 1$, doubled on multilevel meshes, and checked the shift against `maxjshift`. For the 64H SC14 box on 8^3 blocks that is $4 + 30 = 34$ rows, so the pack read far outside the 8-row block (past the end of the array for the last block). The results were not affected, because the actual shift stayed below one cell and those rows were never used. Buffers are now at most one block deep (`nrow` $\le n_{x2}$), about $4\times$ smaller for 8^3 blocks, and the guard compares the shift with what the neighbor can actually supply.

6 Policy and control flow

6.1 The refinement policy

- **I1** (one uniform level along both shear-periodic x_1 faces) is unchanged.
- **I2** (complete x_2 rings) is checked in `Mesh::CheckShearingBoxRefinement` only with `<shearing_box> oa_annular = true`; the default is `false`.
- The **ring-wise flag synchronization** in `MeshRefinement::CheckForRefinement` runs only with `oa_annular = true`. User criteria such as the `gravito_turb` hold/buffer criterion then refine what they flag.
- The **graded cap** and the x_1-boundary veto are unchanged.

6.2 Task sequence of the last RK stage

Task	With orbital advection
`RKUpdate`, `HydroSrcTerms`	as before
`SendU_OA`	`PackAndSendCC`: same / restricted / coarse-box messages
`RecvU_OA`	`RecvAndUnpackCC`: wait, guard, load + prolongate, remap, store G
`SendFlx_OA`	`SendFluxCC`: restricted G to coarser x_2 neighbors
`RecvFlx_OA`	`RecvAndCorrectFluxCC`: replace the face flux on coarse blocks
`RestrictU`, `SendU`, ...	as before

The MHD task list calls the same `OrbitalAdvectionCC` for the cell-centered variables, with the same two flux tasks, and adds two for the face fields (§7).

6.3 A test criterion: the moving box

The shwave problem generator's prescribed moving-ring criterion (`MovingRingRefine`) takes an optional x_2 window, $y_c(t) = y_{c0} + A_y \sin(\omega_y t) + v_y t$, wrapped periodically into $[x_{2,\min}, x_{2,\max})$. With `ring_yhwidth` > 0 a block is refined only if it overlaps both the x_1 interval and the x_2 window (or one of its periodic images): a box that moves in x_1 and drifts through x_2.

7 Magnetic fields: MHD on rings

7.1 How the face-field remap works

`OrbitalAdvectionFC` (`orbital_advection_fc.cpp`) shifts the face-centred field with constrained transport (CT) instead of remapping it directly. For each x_2 column of B_3 (at x_1 cell centres) and of B_1

(at x_1 faces) it builds the extended column from the x_2 neighbours, computes the remap flux exactly as for a cell-centred variable, and turns it into an effective EMF on the x_2 faces, in units of Δx_2:

$$\mathcal{E}_1 = -\left(F + \textstyle\sum_{\text{whole cells}} B_3\right), \qquad \mathcal{E}_3 = +\left(F + \textstyle\sum_{\text{whole cells}} B_1\right), \tag{9}$$

(`emfx` on x_1 edges, `emfz` on x_3 edges; the remap has no $\mathcal{E}_2$). The field is then updated by the discrete curl,

$$B_1 \mathrel{-}= \delta_2\mathcal{E}_3, \qquad B_2 \mathrel{+}= \tfrac{\Delta x_2}{\Delta x_1}\delta_1\mathcal{E}_3 - \tfrac{\Delta x_2}{\Delta x_3}\delta_3\mathcal{E}_1, \qquad B_3 \mathrel{+}= \delta_2\mathcal{E}_1, \tag{10}$$

which keeps $\nabla\cdot B$ unchanged inside every block.

7.2 What goes wrong at a level jump, even with rings

With rings every x_2 neighbour is at the block's level, so the extended columns are the uniform ones; but the blocks still meet coarser or finer blocks across x_1 and x_3 faces. AthenaK keeps both copies of a face shared by two blocks: the regular exchange never overwrites a block's own boundary face (`buffs_fc.cpp`: same-level and finer data start one face beyond it). The copies stay equal only because the stage EMFs on the shared edges are made identical (`RecvAndUnpackFluxFC`: averaged between same-level blocks, replaced by restricted fine values on coarse blocks). The remap's EMFs had no such step. A fine block remaps its fine B_1 on a shared x_1 face, the coarse block remaps its coarse B_1 on the same face, and the two results differ at truncation level: each block stays divergence-free, but the coarse copy drifts away from the mean of the fine copies ($\sim 1\%$ of the field in the tests). That was the unanalysed $\nabla\cdot B$ question behind the old fatal error.

7.3 The correction

With mesh refinement `RecvAndUnpackFC` no longer updates B. It stores the EMFs as $E\,\Delta t$ on the edges (`eoa`: $\mathcal{E}_1\Delta x_2$ in `x1e`, $\mathcal{E}_3\Delta x_2$ in `x3e`, zero `x2e`) and returns. Two new MHD tasks follow `RecvB_OA`:

- `SendE_OA` → `SendEMF_OA`: `PackAndSendFluxFC` on `eoa`;
- `RecvE_OA` → `RecvEMFAndUpdateB_OA`: `RecvAndUnpackFluxFC` on `eoa`, conversion back to units of Δx_2, and the CT update (`UpdateB`).

The exchange is the stage's own EMF machinery, run on a private `MeshBoundaryValuesFC` instance (`pbval_oa`, with its own buffers and MPI communicators), so it cannot collide with the stage's EMF messages, whose sends are still in flight. It handles faces and edges, same-rank copies and MPI, and skips the shear-periodic x_1 boundary as the stage correction does. Physical units are needed because the correction averages EMFs along edges: a coarse edge carries the mean of its two fine edges, which in units of the block's own Δx_2 would carry a factor 1/2. Receives are posted in `OrbitalAdvectionFC::InitRecv` and cleared with the base buffers. Without refinement nothing of this runs, and the arithmetic is the old one.
Two further changes in the class:

- **Buffer depth.** B_1, B_3 rows sent across an x_2 face: `nrowb` $= \min(n_{x2},\ \max(\texttt{ng}+\texttt{maxjshift}+1,\ \texttt{oa_nrow}))$, at least the cell-centred `nrow` and never more than one block (the old depth `ng` + `maxjshift` could exceed the block).
- **EMF arrays** `emfx`, `emfz` are sized for the largest pack AMR can produce (the larger of the pack size and `nmb_maxperrank`), as the other MHD arrays are; they were sized for the initial pack, which was harmless only while refinement was fatal.

7.4 Policy

`MHD::MHD` accepts orbital advection on a refined mesh when `<shearing_box> oa_annular = true`, which makes `Mesh::CheckShearingBoxRefinement` require complete x_2 rings and synchronizes AMR flags ring-wise. Without it the error names the option. The cell-centred flux tasks (`SendFlx_OA`, `RecvFlx_OA`) are in the MHD task list as well; on rings they do nothing.

7.5 A diagnostic: shared-face consistency

The swing problem generator's history has two more columns with MHD. After the boundary exchange a fine block holds, in `coarse_b0`, the coarse neighbour's copy of every face they share; `bmis` $= \sum |B_n^{\mathrm{coarse}} - \langle B_n^{\mathrm{fine}} \rangle_{2\times 2}| \, \Delta A$ over the six faces of every fine block with a coarser neighbour, and `bnrm` $= \sum |B_n| \, \Delta A$ over the same faces. Their ratio is round-off when the copies agree.

8 Input-file interface

8.1 Parameters

Block	Parameter	Default	Meaning
`<shearing_box>`	`oa_annular`	`false`	`true`: complete x_2 rings required (I2) and AMR flags synchronized ring-wise
	`oa_flux_correct`	`true`	Orbital-advection flux correction at x_2 level jumps (`false` for tests only)
	`oa_nrow`	$\min(n_{x2},$ `maxjshift` $+3)$	Rows sent across a same-level x_2 face; at most n_{x2}
`<problem>` (shwave)	`ring_yc0`, `ring_yamp`	0	Window center and oscillation amplitude
	`ring_yfreq`, `ring_yvel`	1, 0	Oscillation frequency and drift speed
	`ring_yhwidth`	-1	> 0: refine only inside the x_2 window

Command-line overrides replace only parameters already in the input file, so `oa_annular` and `oa_flux_correct` must be written into a copy of the input when they are to be changed and are not there.

8.2 Reference inputs

File (under `inputs/`)	Configuration
`shearing_box/epicycle_smr_ypatch.athinput`	Epicycle, level-1 patch over part of x_2
`shearing_box/hydro_incompress_shwave_smr_ypatch.athinput`	Vortical shwave, static patch
`shearing_box/hydro_incompress_shwave_amr_box.athinput`	Vortical shwave, moving box (AMR)
`tests/swing_selfgrav_smr_ypatch.athinput`	Swing with multigrid gravity, patch

9 Guard reference

Location	Condition	Protects
`OrbitalAdvectionCC::RecvAndUnpackCC`	integer shift $+3 >$ rows available from the x_2 neighbors	The remap stencil stays inside the immediate neighbor; normally prevented by the timestep limit
`Mesh::CheckShearingBoxRefinement`	incomplete x_2 ring with `oa_annular = true`	The optional annular policy
`Mesh::CheckShearingBoxRefinement`	x_1-face blocks on more than one level	I1 (unchanged)
`MHD::MHD`	orbital advection + refinement without `oa_annular = true`	Face fields across x_2 level jumps (not implemented yet)

10 Limitations and next steps

1. **MHD across x_2 level jumps.** Rings work (§7). With partial-x_2 refinement the face-field remap also needs the neighbour's B_1, B_3 at the block's resolution (restriction from finer, divergence-preserving prolongation from coarser) and the EMF correction on x_2 faces and edges.

2. **One neighbor deep.** All upstream data come from the immediate x_2 neighbor; the timestep limit enforces it. Reaching through two neighbors would lift the limit on small blocks far from $x = 0$.

3. **Previous-stage ghosts in the coarse box.** They enter only the limited slopes of the prolongation; a separate orbital-advection stage with its own boundary cycle (as in Athena++) would remove this at the cost of an extra exchange.

4. **Non-FARGO refinement with x_2 level jumps** collapses at the first step ($\Delta t \to 10^{-45}$) — in the parent code too, with or without `prolong_primitives`. It is outside this work.

5. **Production test pending.** The SC14 clump-following AMR with and without rings (block count and cost) has not been run yet.